

Bigdata 101

Mapreduce ve Yarn

Veysel Yüksel



Table of Contents

01

Mapreduce Nedir

02

Mapreduce Mimari

03

Mapreduce vs Spark

04

Mapreduce Örnek

05

YARN

06

YARN Mimari

Mapreduce Nedir?

Dağıtık mimaride depolanan büyük veri setlerinin paralel ve dağıtık işlem mantığı ile işlenmesini sağlayan bir framework'tür. Temel özellikleri;

- Paralel İşlem
- Dağıtık Mimari
- Map & Reduce Modeli
- Disk Tabanlı İşlem
- HDFS ile Entegre
- Hata Toleransı
- Data Locality
- YARN üzerinde çalışır
- Güvenilirlik

Mapreduce Nedir?

Avantajları;

- Yüksek Hata Toleransı
- Büyük Veriye Uygun
- Ölçeklenebilir
- Veri Yakınlığı
- Otomatik Task Yönetimi
- Açık Kaynak & Geniş Destek

Dezavantajları;

- Yavaş
- Gerçek Zamanlı Değil
- Karmaşık Kodlama
- Küçük İşler İçin Aşırı
- Interaktif Değil

Mapreduce Nedir?

Ne zaman kullanılmalı?

- Büyük ve sabit yapılı veri setlerini batch (toplu) işlemek gerekiyorsa.
- Gerçek zamanlılık gerekmeyen durumlarda
- Hadoop cluster zaten kuruluysa ve Spark/Flink gibi alternatifler yapılandırılmamışsa
- Tarihsel log analizi, ETL işlemleri gibi işler için hâlâ tercih edilebilir

MapReduce'e Uygun Workload Örnekleri



Kelime Sayımı

Metin koleksiyonunda her kelimenin frekansını hesaplar



Web Log Analizi

Web loglarını analiz eder, trafik ve hata oranlarını özetler



Clickstream Analizi

Kullanıcıların site içi davranışlarını inceler



ETL İşlemleri

Veriyi dönüşüm işlemleriyle düzenler ve depolar



Finansal Özetleme

Finansal verileri toplar ve özetler



Tersine İndeksleme

Arama motorları için kelimedenden belgeye indeksleme yapar



Genom Verisi Analizi

Genom dizilerindeki örüntüleri bulur

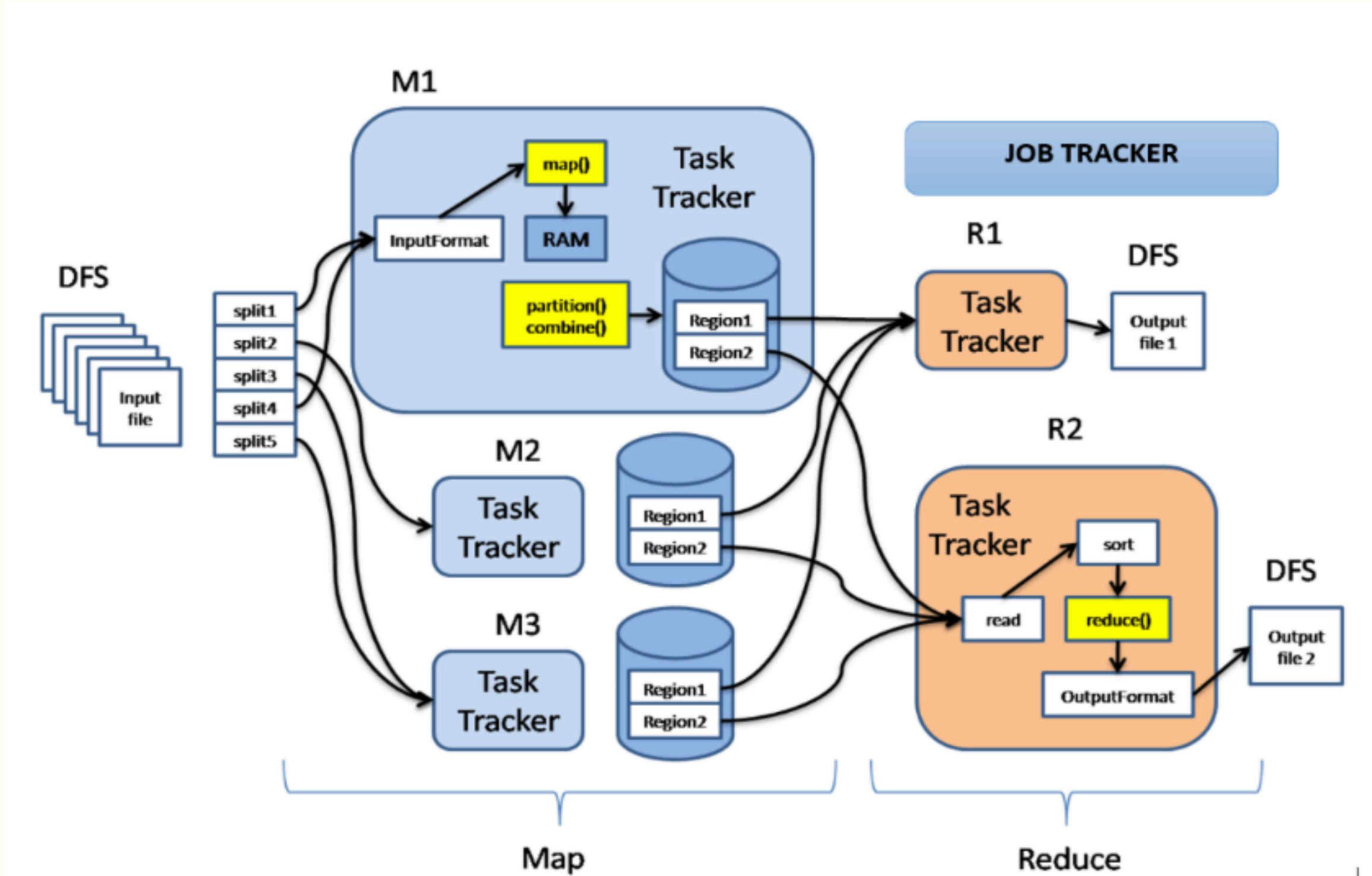


Kümeleme

Benzer veri noktalarını gruplar

Mapreduce - Mimari

Map ve reduce isimli iki temel aşamadan oluşur. Map aşamasında veri bölme, Reduce aşamasında veri birleştirme operasyonları gerçekleşir.



Mapreduce - Mimari

JobTracker

- MapReduce Job'ları için kaynak yönetimi, zamanlama ve izleme gibi görevleri yerine getirir.
- Aynı zamanda, NameNode'lar ile iletişim kurarak veri setinin lokasyonunu belirler.

Task Tracker

- Esas işi yapan kısımdır.
- Map ve Reduce fazları arasında Intermediate Process adı verilen bir işlem bulunur.
- Intermediate Process map fazından sonra verinin sıralanması işlemini yapar.
- Job Tracker – Task Tracker arasında master-slave ilişkisi bulunur. Job Tracker NameNode üzerinde çalışır, Task Tracker'lar ise DataNode'lar üzerinde çalışır ve birden çok sayıdadır.

Mapper



- Mapper işlemi MapReduce sürecinin ilk fazıdır.
- Mapper task, input verilerini işleyerek bu veri üzerinde key-value değerleri üretir.
- Bu faz sonucunda çıkan output ise üretilmiş olan key,value değerlerinin sıralı bir bütünüdür.
- MapReduce işlemi sırasında her InputSplit için bir tane Map task'i çalıştırılır.
- InputSplit, verinin mantıksal olarak parçalanmış şeklidir ve Mapper tarafından işlenen verileri temsil eder.
- RecordReader ise InputSplit ile iletişim kurarak, verinin Mapper tarafından okunabilmesi için key,value formatına dönüştürülmesini sağlar.
- Mapper sayısı = Toplam veri boyutu / InputSplit boyutu formülü ile hesaplanır.

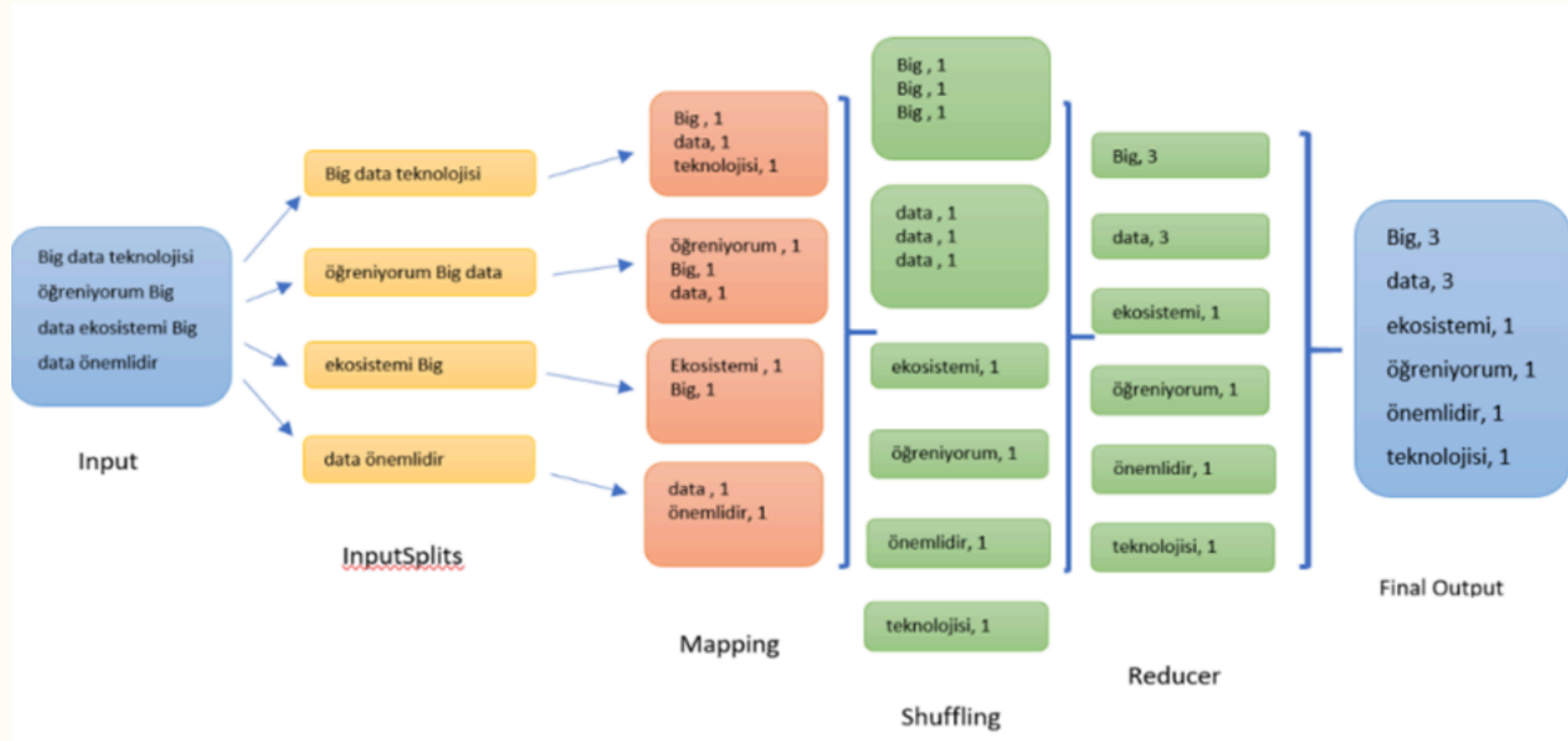
Reducer



- Reducer, Mapper işleminden elde edilen sonucu input verisi olarak alır, işler ve sonuç veri setini oluşturur.
- Reduce işlemi sonucunda elde edilen veri HDFS'te depolanır.
- Reducer aggregation denilen, Mapper'den elde edilen sonuçların birleştirilmesi işini yapar.
- Reducer işlemi, shuffle fazı, sort fazı ve reduce fazı olmak üzere kendi içinde üç faza ayrılır.
- Shuffle fazında mapper işleminin output verisinin Reducer işlemine input olarak iletilmesi işlemi gerçekleşir.
- Sort fazında mapper işlemi sonucu elde edilen key-value verisi, key değerine göre sıralanır.

Mapreduce Örnek

HDFS üzerinde yer alan bir text dosyasının içerisindeki kelimeleri saydıran bir MapReduce işlem adımları şu şekilde gerçekleşir.



Mapreduce vs Spark

- MapReduce, belirli sınırlamalar ve performans kısıtlamalarıyla birlikte gelir. Karmaşık veri işleme senaryolarında, yinelemeli algoritmalarda ve yüksek düzeyli işlemlerde bu sınırlamalar daha belirgin hale gelir.
- Bu nedenlerle, modern veri işleme ihtiyaçları için Apache Spark gibi alternatifler daha hızlı performans, esneklik ve gelişmiş işlem yetenekleri sunarak sıklıkla tercih edilmektedir.
- Spark, bellek içi işleme ve daha geniş bir işlem modeli sunarak bu sınırlamaları aşmayı hedefler

Mapreduce vs Spark

Mapreduce

- Uygun olduğu durumlar: Büyük ölçekli veri kümesi işleme, büyük veri analizi, verilerin basit şekilde işlenmesi.
- Kullanım: Büyük veri işlemeyi anlamlı bir şekilde gerçekleştirmek için hala kullanılabilir, ancak daha modern çözümlerle genellikle birlikte kullanılmaz.
- Real time senaryolarda fail eder.

Spark

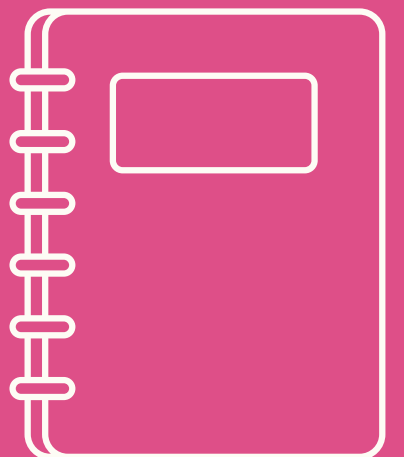
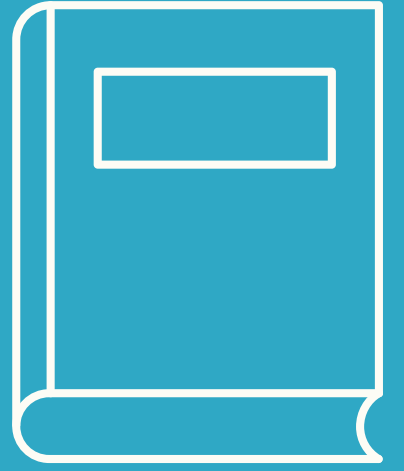
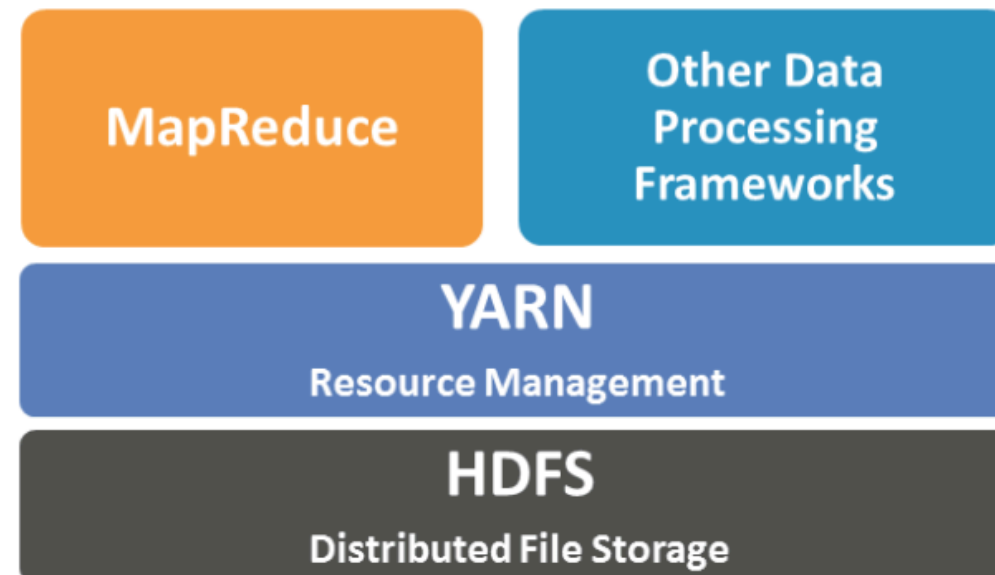
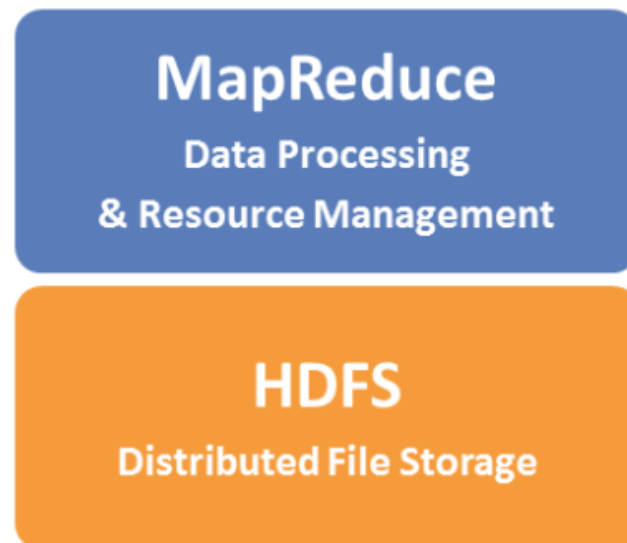
- Uygun olduğu durumlar: Hızlı veri işleme, gerçek zamanlı veri analizi, karmaşık sorgular ve veri işleme senaryoları.
- Kullanım: Daha hızlı ve çeşitli veri işleme ihtiyaçları için tercih edilir, özellikle yüksek performans gerektiren durumlarda.
- Real time data analizi gereken durumlarda da çalışır.

Mapreduce vs Spark

Factors	Spark	Hadoop MapReduce
Speed	100x times than MapReduce	Faster than traditional system
Written In	Scala	Java
Data Processing	Batch / real-time / iterative / interactive / graph	Batch processing
Ease of Use	Compact & easier than Hadoop	Complex & lengthy
Caching	Caches the data in-memory & enhances the system performance	Doesn't support caching of data

YARN

YARN (Yet Another Resource Negotiator), Hadoop 2.x ve sonrası sürümlerde bulunan kaynak yönetim sistemidir. Hadoop ekosisteminin merkezinde yer alır ve MapReduce, Spark, Hive gibi farklı veri işleme motorlarının aynı cluster üzerinde çalışmasını sağlar.



YARN Temel Özellikler



- Kaynak Yönetimi
 - YARN, kaynakları dinamik olarak tahsis eder ve iş yüklerinin kaynak taleplerine göre kaynakları yönetir
- Çeşitli İşleme Motorları Desteği
 - YARN, sadece MapReduce değil, aynı zamanda Apache Spark, Apache Tez, Apache Flink gibi diğer veri işleme motorları ile uyumlu çalışır.
- Ölçeklenebilirlik
 - YARN, büyük veri kümeleri ve büyük ölçekli küme yapıları için ölçeklenebilirlik sunar.
- Gelişmiş Planlama
 - YARN, iş yüklerinin kaynaklara göre daha iyi planlanmasını ve denetlenmesini sağlar.
- Dinamik Kaynak Tahsisi
 - YARN, iş yüklerinin ihtiyaçlarına göre kaynakları dinamik olarak tahsis eder.

YARN Mimari

YARN aşağıdaki bileşenlerden meydana gelir.

- Resource Manager
- Node Manager
- Application Master
- Container

Resource Manager, YARN'ın temel bileşenidir. Görevi isminden anlaşılacağı gibi, kaynakların yönetimidir.

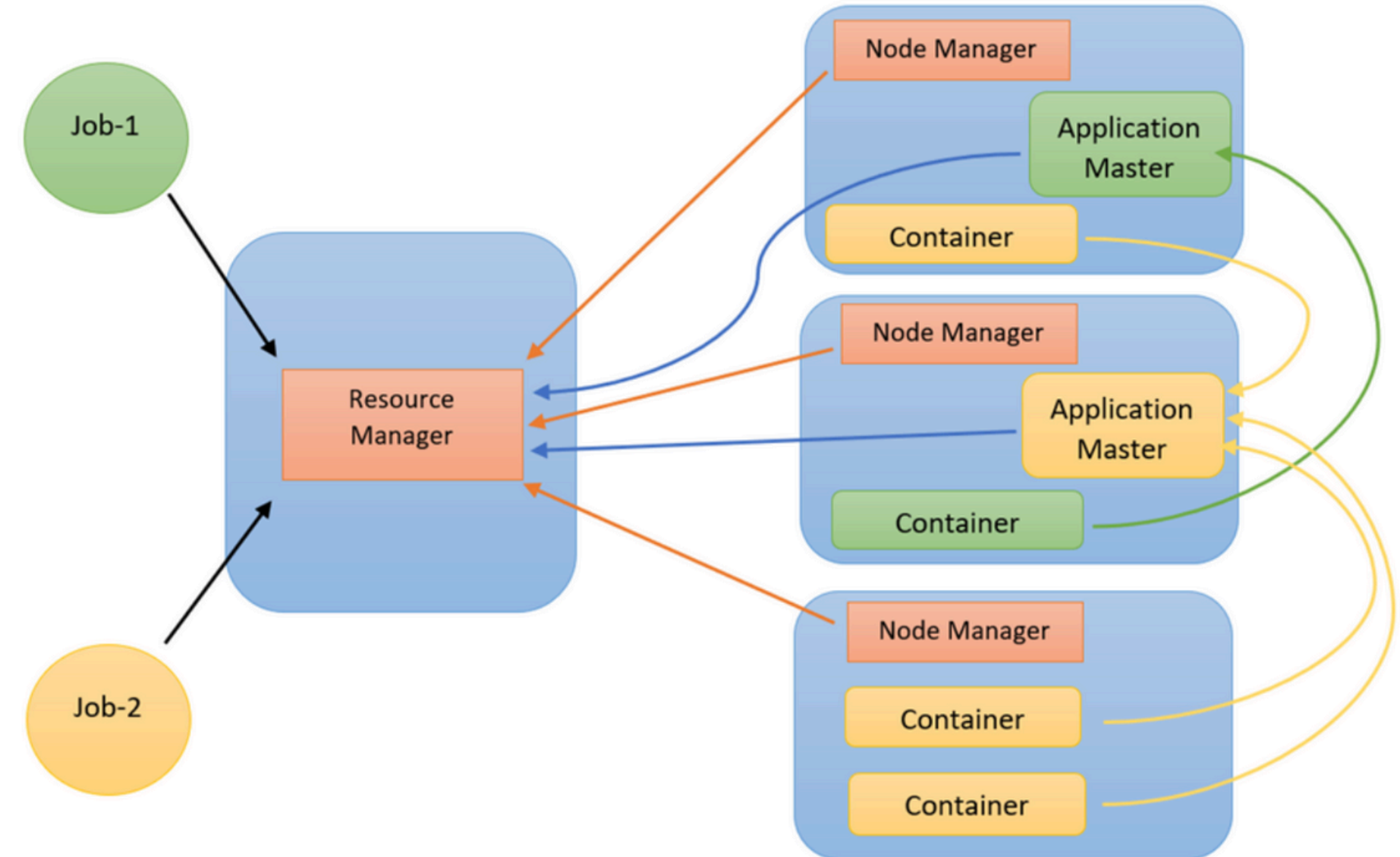
Node Manager, YARN'ın node'lar üzerinde çalışan ve üzerinde çalıştığı node'un kaynaklarını izleyen bileşenidir.

- **Application Master**, Cluster'da yapılacak işin koordinasyonunu sağlar. Her işlem için bir Application Master bileşeni çalışır.
- Container, Bir node üzerindeki fiziksel kaynakların (CPU, RAM ve Disk) birleşimi Container'ı oluşturur. Bir node üzerinde, gelen talep için ayrılan spesifik kaynakların kullanımını sağlar.

YARN Mimari

- Job-1 ve Job-2 işlemleri Resource Manager tarafından kabul edilir.
- Cluster'da her işlem için ayrı bir Application Master işlemi bulunur.
- Application Master'lar kendilerine ait Container'lar ile iletişim kurarken; her Application Master da Resource Manager ile iletişim kurup kullanabileceği kaynakların (Mavi oklar) bilgisini alır.
- Container'lardan, Application Master'lara MapReduce işleminin durum bilgisi (Sarı ve yeşil oklar) iletilir.
- Node Manager'lardan Resource Manager'a Node'ların durum bilgisi (Turuncu oklar) iletilir. Sırası ile;
 - Kullanıcıdan bir iş talebi gelir.
 - Resource Manager Application Manager'ı başlatmak için bir Container ayırır.
 - Application Manager'ın bilgisi Resource Manager'a kaydedilir.
 - Application Manager Node üzerinde bir Container oluşturmak için Resource Manager'a kaynak bilgisi sorar.

- Application Manager Node Manager'a container'ı oluşturması için bilgi verir.
- Gerçekleştirilmesi istenen işlem Container'ın içinde çalıştırılır.
- Kullanıcı talep ettiği işlemin durumunu izlemek için Resource Manager ile iletişim kurar.
- İşlem tamamlandığında, Application Manager'ların Resource manager'dan kaydı silinir.



YARN Nerde Kullanılır



- Hadoop MapReduce job'larında
- Apache Spark uygulamalarında
- Apache Hive, Tez, Flink, HBase gibi sistemlerde
- Gerçek zamanlı ve batch işlerde kaynak yönetiminde

YARN Avantajları

Çoklu İş Desteği	Aynı anda birçok framework çalışabilir
Yüksek Kaynak Verimi	Dinamik kaynak paylaşımı yapılır
İzlenebilirlik	Her job'un durumu merkezi olarak izlenir
Ölçeklenebilirlik	Yüzlerce-node ölçeğinde çalışabilir
Yedeklilik	ResourceManager High Availability (HA) desteği vardır

Soru-Cevap

